INTERVIEW PREP SERIES

# Next.js

## Interview Questions & Easy Answers

---

**52 questions · 22 marked ★ most asked**

Covers the App Router, file-based routing, Server and Client Components, rendering strategies (SSR, SSG, ISR, CSR), data fetching and caching, Server Actions, route handlers, SEO, images, auth and deployment. Based on current (App Router) interview question sets.

### How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

# Basics

### Q1. What is Next.js? ★ Most asked

Next.js is a React framework made by Vercel. React only builds the UI; Next.js adds everything else needed for a production app: file-based routing, server-side rendering, static generation, API routes, image and font optimisation, and SEO features.

### Q2. What is the difference between React and Next.js? ★ Most asked

- React is a **library** for UI; Next.js is a **framework** built on React.
- React apps (Vite/CRA) render on the client by default; Next.js can render on the server (better SEO and faster first load).
- React needs React Router; Next.js has built-in file-based routing.
- Next.js can also write backend endpoints (route handlers) in the same project.

### Q3. What are the advantages of Next.js? ★ Most asked

Better SEO with SSR/SSG, faster first load, file-based routing, built-in API routes, automatic code splitting, image optimisation with `next/image`, easy deployment on Vercel, and support for TypeScript out of the box.

### Q4. How do you create a Next.js project?

`npx create-next-app@latest my-app`. It asks about TypeScript, ESLint, Tailwind and the App Router, then creates the project. Run it with `npm run dev`.

# Routing

### Q5. What is the App Router? How is it different from the Pages Router? ★ Most asked

The App Router (the `app/` folder, Next.js 13+) is the modern routing system. It uses React Server Components by default, supports nested layouts, loading and error files, and Server Actions. The older Pages Router (the `pages/` folder) uses `getServerSideProps` and `getStaticProps` for data. New projects should use the App Router.

### Q6. What is file-based routing? ★ Most asked

The folder structure decides the URLs. In the App Router, each folder is a route segment and `page.tsx` makes it a visible page.

```
app/page.tsx              -> /
app/about/page.tsx        -> /about
app/blog/[slug]/page.tsx  -> /blog/my-first-post
```

### Q7. What is the difference between `page.tsx`, `layout.tsx` and `template.tsx`? ★ Most asked

- `page.tsx`: the unique UI of a route; makes the route accessible.
- `layout.tsx`: shared UI (navbar, sidebar) that wraps pages and **keeps its state** when you navigate between child pages. `app/layout.tsx` is the required root layout with `<html>` and `<body>`.
- `template.tsx`: like a layout but **re-mounts** on every navigation, so state resets.

### Q8. What are dynamic routes? ★ Most asked

Routes with a variable part, created with square brackets.

```
// app/products/[id]/page.tsx
```

```
export default async function Product({ params }) {
  const { id } = await params;   // params is a promise in Next 15+
  return <h1>Product {id}</h1>;
}
```

`[...slug]` catches all segments (`/docs/a/b/c`) and `[[...slug]]` is an optional catch-all.

### Q9. What are route groups and private folders?

Route groups `(name)` organise routes without adding to the URL, e.g. `app/(auth)/login/page.tsx` is just `/login`. Private folders `_name` are ignored by routing and hold helper components.

### Q10. What are `loading.tsx`, `error.tsx` and `not-found.tsx`? ★ Most asked

- `loading.tsx` shows a loading UI (a skeleton or spinner) automatically while the page loads, using React Suspense.
- `error.tsx` is an error boundary that shows a fallback UI if something fails in that segment. It must be a Client Component and receives `error` and `reset`.
- `not-found.tsx` shows a custom 404 page, also triggered by calling `notFound()`.

### Q11. How do you navigate between pages? ★ Most asked

Use the `<Link>` component for client-side navigation without full reloads; it also prefetches pages.

```
import Link from "next/link";
<Link href="/about">About</Link>
```

In code, use `useRouter()` from `next/navigation` in a Client Component: `router.push("/dashboard")`.

### Q12. What is the difference between `redirect()` and `router.push()`?

`redirect("/login")` from `next/navigation` is used on the server (Server Components, Server Actions, route handlers) and sends the user to a new URL before the page renders. `router.push()` is client-side navigation triggered by user actions in a Client Component.

### Q13. What are `useParams`, `useSearchParams` and `usePathname`?

Client hooks from `next/navigation`. `useParams()` reads dynamic segments, `useSearchParams()` reads `?query=` values, and `usePathname()` gives the current path (useful for highlighting the active nav link).

## Server and Client Components

### Q14. What are Server Components? ★ Most asked

In the App Router, every component is a Server Component by default. It runs only on the server, can be `async`, can fetch data or read the database directly, and sends ready HTML to the browser with no extra JavaScript. Benefits: smaller bundles, faster pages, and secrets stay on the server.

### Q15. What are Client Components and when do you use `"use client"`? ★ Most asked

Client Components run in the browser (and are pre-rendered on the server). You mark a file with `"use client"` at the top when you need:

- State and effects: `useState`, `useEffect`
- Event handlers: `onClick`, `onChange`
- Browser APIs: `window`, `localStorage`

- Hooks like `useRouter` or other client-only libraries

**Interview tip:** Say: "Keep components on the server by default, and push `use client` down to the small interactive parts, like a button or a form."

### Q16. Can a Server Component import a Client Component? And the opposite?

A Server Component can import and render a Client Component. A Client Component cannot import a Server Component directly, but it can receive one as `children` or as a prop from a Server Component parent.

### Q17. What is hydration?

The server sends HTML so the page shows quickly; then React loads the JavaScript in the browser and attaches event handlers to that HTML. This "making it interactive" step is hydration.

### Q18. What causes hydration errors and how do you fix them? ★ Most asked

A hydration error happens when the HTML rendered on the server doesn't match what React renders in the browser. Common causes: using `Date.now()`, `Math.random()`, `window` or `localStorage` during render, or invalid HTML nesting (a `<div>` inside a `<p>`). Fix: move browser-only code into `useEffect`, or load the component with `dynamic(() => import(...), { ssr: false })`.

## Rendering and data fetching

### Q19. What is the difference between SSR, SSG, ISR and CSR? ★ Most asked

- **SSG (Static Site Generation)**: HTML is built once at build time and served from a CDN. Fastest. For blogs and docs.
- **SSR (Server-Side Rendering)**: HTML is built on every request. Always fresh. For dashboards and personalised pages.
- **ISR (Incremental Static Regeneration)**: static pages that rebuild in the background after a time limit. For product pages.
- **CSR (Client-Side Rendering)**: the browser builds the page with JS after loading. For highly interactive, private pages.

### Q20. How do you fetch data in the App Router? ★ Most asked

Make the Server Component `async` and use `await` directly.

```
export default async function Posts() {
  const res = await fetch("https://api.example.com/posts");
  const posts = await res.json();
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

You can also query the database directly here. No `useEffect` needed.

### Q21. How do you control static vs dynamic rendering?

A route is static if it uses no request-time data. It becomes dynamic when it uses `cookies()`, `headers()`, `searchParams`, or uncached fetches. You can also force it with `export const dynamic = "force-dynamic"` or choose time-based revalidation with `export const revalidate = 60`.

### Q22. What is ISR and how do you use it? ★ Most asked

ISR lets a static page update after deployment without rebuilding the whole site.

```
fetch(url, { next: { revalidate: 60 } }); // refresh at most once a minute
```

The first visitor after 60 seconds gets the cached page while Next.js regenerates it in the background. You can also trigger updates on demand with `revalidatePath("/blog")` or `revalidateTag("posts")`.

### Q23. What is `generateStaticParams()`?

Used with dynamic routes to tell Next.js which pages to build at build time (SSG).

```
export async function generateStaticParams() {
  const posts = await getPosts();
  return posts.map(p => ({ slug: p.slug }));
}
```

### Q24. How does caching work in Next.js at a high level?

Next.js can cache fetched data, rendered pages and client-side navigations so it doesn't redo work. Defaults have changed between versions (Next 15 made fetch uncached by default, and Next 16 added the `"use cache"` directive). Key point: you choose what is cached and refresh it with `revalidate`, `revalidatePath` or `revalidateTag`.

### Q25. What is streaming?

Instead of waiting for all data, Next.js sends parts of the page as soon as they're ready. Wrap slow components in `<Suspense fallback={...}>` (or use `loading.tsx`) so the rest of the page shows immediately.

## Server Actions and API

### Q26. What are Route Handlers? ★ Most asked

Route Handlers let you build API endpoints inside the `app` folder using `route.ts` files.

```
// app/api/users/route.ts
export async function GET() {
  return Response.json([{ id: 1, name: "Asha" }]);
}
export async function POST(request: Request) {
  const body = await request.json();
  return Response.json(body, { status: 201 });
}
```

In the Pages Router, the equivalent is `pages/api/*.js`.

### Q27. What are Server Actions? ★ Most asked

Server Actions are async functions marked with `"use server"` that run on the server but can be called directly from forms or components, without writing an API route.

```
async function addTodo(formData: FormData) {
  "use server";
  await db.todo.create({ data: { title: formData.get("title") } });
  revalidatePath("/todos");
}
<form action={addTodo}><input name="title" /><button>Add</button></form>
```

### Q28. What is the difference between a Route Handler and a Server Action?

A Route Handler is a public HTTP endpoint (GET/POST etc.) that any client, like a mobile app, can call. A Server Action is a function for mutations from your own Next.js UI, especially forms. Use Route Handlers for webhooks and external clients, and Server Actions for in-app form submissions.

### Q29. What is middleware (proxy) in Next.js? ★ Most asked

Code that runs before a request reaches a route, used for auth checks, redirects, rewrites and locale detection. It was `middleware.ts` until Next.js 15 and was renamed to `proxy.ts` in Next.js 16. It lives at the project root and can use a `matcher` to choose which paths it runs on.

### Q30. How do environment variables work in Next.js?

Put them in `.env.local`. Variables are server-only by default. Only variables starting with `NEXT_PUBLIC_` are exposed to the browser, e.g. `NEXT_PUBLIC_API_URL`. Never put secrets in `NEXT_PUBLIC_` variables.

## Optimisation and SEO

### Q31. How do you add SEO metadata in Next.js? ★ Most asked

Export a `metadata` object or a `generateMetadata` function from a `layout` or `page`.

```
export const metadata = {
  title: "My Store",
  description: "Buy great products",
};

export async function generateMetadata({ params }) {
  const { id } = await params;
  const p = await getProduct(id);
  return { title: p.name, description: p.summary };
}
```

Next.js also supports `sitemap.ts`, `robots.ts` and Open Graph images.

### Q32. What is `next/image` and why use it? ★ Most asked

The `<Image>` component optimises images automatically: resizes them for each device, serves modern formats like WebP/AVIF, lazy-loads them, and prevents layout shift because width and height are required.

```
import Image from "next/image";
<Image src="/hero.png" alt="Hero" width={800} height={400} priority />
```

### Q33. What is `next/font`?

It loads Google or local fonts at build time and self-hosts them, removing extra network requests and layout shift. `const inter = Inter({ subsets: ["latin"] });` then `className={inter.className}`.

### Q34. What is `next/dynamic`?

It lazy-loads a component only when needed, reducing the initial bundle. With `{ ssr: false }` it skips server rendering for browser-only libraries (like charts or maps).

### Q35. What is automatic code splitting?

Next.js splits JavaScript per route, so a user visiting `/about` downloads only the code for that page, not the whole app.

### Q36. How does Next.js help with Core Web Vitals?

Server rendering improves LCP (largest contentful paint), `next/image` and `next/font` reduce CLS (layout shift), and smaller client bundles with Server Components improve INP (interaction responsiveness).

## Auth, deployment and Pages Router

### Q37. How do you handle authentication in Next.js? ★ Most asked

Use a library like **Auth.js (NextAuth)**, Clerk, or your own JWT/session cookies. Store tokens in httpOnly cookies, check the session in Server Components or route handlers, and protect routes with the proxy/middleware that redirects unauthenticated users to `/login`. Always re-check auth inside Server Actions too.

### Q38. What are `getServerSideProps` and `getStaticProps`?

Data-fetching functions in the old **Pages Router**. `getStaticProps` runs at build time (SSG), `getServerSideProps` runs on every request (SSR), and `getStaticPaths` lists dynamic pages to pre-build. In the App Router, you use async Server Components instead.

### Q39. How do you deploy a Next.js app?

The easiest way is **Vercel**: connect the GitHub repo and it deploys on every push. You can also run `next build` and `next start` on any Node server (AWS EC2, Render), use Docker, or use `output: "export"` for a fully static site.

### Q40. What is `next.config.js` used for?

It configures the app: allowed image domains (`images.remotePatterns`), redirects and rewrites, environment settings, and experimental features.

### Q41. What is the `public` folder?

Files in `public/` are served from the root URL as-is: `public/logo.png` becomes `/logo.png`. Used for images, favicons and `robots.txt`.

### Q42. How do you style a Next.js app?

Global CSS imported in the root layout, CSS Modules (`styles.module.css`) for scoped styles, Tailwind CSS (most popular), or Sass. CSS-in-JS libraries need extra setup with Server Components.

### Q43. What is Turbopack?

Turbopack is the Rust-based bundler made by Vercel to replace Webpack in Next.js. It makes the dev server start and update much faster, and it is the default bundler in recent versions.

### Q44. What are parallel and intercepting routes?

Parallel routes (`@slot` folders) show multiple pages in the same layout at once, like a dashboard with separate panels. Intercepting routes (`(.)photo`) let you show a route in a modal (like opening a photo over a feed) while keeping a real URL.

### Q45. Scenario: a product page shows a stale price. How would you fix it?

The page is probably cached as static. Either set a short `revalidate` time, call `revalidateTag`/`revalidatePath` when the price is updated, or make the price part dynamic by fetching with `cache: "no-store"`.

### Q46. Scenario: you need an "Add to Cart" button on a Server Component page.

Keep the page as a Server Component and create a small `AddToCartButton` Client Component with `"use client"` for the click handler. Import it into the page. Optionally the button calls a Server Action to update the cart.

### Q47. Scenario: reading the theme from localStorage causes a hydration mismatch.

`localStorage` doesn't exist on the server, so the server and client render differently. Read it inside `useEffect` after mount (show a default first), or store the theme in a cookie so the server can read it too.

## More frequently asked

### Q48. What is the difference between `next dev`, `next build` and `next start`?

`next dev` runs the development server with hot reload. `next build` creates an optimised production build and pre-renders static pages. `next start` runs that production build on a Node server.

### Q49. How do you show a page that reads search params, like `/search?q=phone`?

In a Server Component page, read the `searchParams` prop (`const { q } = await searchParams;`) and fetch results on the server. In a Client Component, use `useSearchParams()`. Reading search params makes the page dynamic.

### Q50. What is the difference between `<Link>` and `<a>` in Next.js?

`<Link>` does client-side navigation (no full reload), keeps shared layouts mounted and prefetches the next page, so navigation feels instant. A plain `<a>` reloads the whole page; use it for external links.

### Q51. What is `"use server"` vs `"use client"`?

`"use client"` marks a file as a Client Component boundary that runs in the browser. `"use server"` marks functions as Server Actions that can be called from the client but always run on the server. They are not opposites for components: components are server by default with no directive.

### Q52. Scenario: how would you protect a `/dashboard` route? ★ Most asked

Check the session cookie in `proxy.ts` (or `middleware.ts` before Next 16) and redirect to `/login` if it's missing. Also verify the session inside the dashboard's Server Component and in any Server Actions or route handlers, because a proxy check alone is not enough for security.